Bug or candidate	Mechanism	Status	Evidence
STT `stt`			
Visibility-point/squash race	Taint clears when the branch resolves, so the wrong-path transmitter issues before the squash arrives	confirmed †	one trace at slow commit timing; none at stock timing
Pending-squash race	An untainted branch's squash waits behind an older tainted one's, so its timing reveals the secret	confirmed	upstream fix not in our build; our gadget does not show it
Implicit-channel defense inert	`hasImplicitFlow` is computed but never reaches the taint check	suspected	code read; our branch audits find no leak (42/42 sound)
Speculative store fills the D-TLB (KV3)	Stores never get the speculative flag, so a wrong-path store walks and fills the D-TLB	not a bug ‡	our gadget shows it; the load control is protected
Excess mispredict-taken hits	6–7× more hits in taken than in not-taken mode, on this build only	untriaged	audit note; predates the checker and harness fixes
SPT-Fence `gem5-spt`			
Same visibility-point/squash race	The fence lifts at branch resolution; the transmitter sends a packet before the squash	strong candidate †	≈3,800 slow-timing hits, none at stock; no branch annotations loaded
Branch-transmitter hits	Transmitter branches redirect inside the window; not yet bucketed	untriaged	post-fix: 28 and 22 tests; 80 earlier hits unexplained
Load-transmitter hits	In-window loads, mixed with the known constant-address model artifact	untriaged	post-fix: 336 and 264 of 1,697 tests
Store-address transmitter hits	In-window stores, scored by the load checker (no store checker yet)	untriaged	24 hits on the regenerated testset
ReCon (STT scheme) `gem5-recon-modded`; the OTB hand-built test ran on `gem5-recon`			
Store-to-load forwarding skips the taint check (STL)	The forwarding path returns before the taint check, so a load with a tainted address completes	confirmed	generated test `inst-014912` with trace; hand-built tests
`getOldestTaint` keeps only the oldest taint (OTB)	An ALU result is untainted once its older source resolves, though a younger source is still speculative	confirmed	hand-built test; 30/30 Alloy-forced gadgets; not enumerated
Load-transmitter hits	The same 7 tests hit in both modes; not checked against the send-line gap	untriaged	post-fix: 7 of 1,323 tests
InvisiSpec `amulet-InvisiSpec`			
Speculative load evicts an L1D line (UV1)	A speculative miss evicts a live L1D line before the load is squashed	confirmed	29/30 tests; AMuLeT's patch removes it
D-TLB hit updates replacement state	A speculative D-TLB hit updates LRU at once; the paper delays it to the visibility point	strong candidate	code read (`tlb.cc:146`); events in our UV1 trace
Directory invalidates on a speculative fetch	A speculative fetch to a line held in M invalidates its owner	suspected	code read ("Is it secure?" comment); not seen in traces
MSHR/TBE contention (UV2)	Speculative misses hold both TBEs, so later accesses stall	not a bug ‡	our gadget shows it (2 vs. 8 TBEs)
Speculative fetch fills L1I (KV1)	A wrong-path fetch installs an L1I line; only data loads are protected	not a bug ‡	our gadget shows it; the control does not
CleanupSpec `amulet-CleanupSpec`			
Split load not cleaned (UV4)	A line-crossing load installs two lines; cleanup undoes neither	confirmed	our gadget; the aligned control is cleaned
Over-cleaning (UV5)	Cleanup evicts a line that a correct-path load installed	AMuLeT-reported, not reproduced	invalidate-before-restore order prevents it in our gadgets
Speculative store not cleaned (UV3)	Cleanup undoes loads only, so a speculative store's cache change would stay	AMuLeT-reported, not reproduced	unreachable here: squashed stores never touch the cache
Cleanup timing reveals hit or miss (unXpec)	Cleanup does 7 events after a speculative miss and none after a hit	not a bug ‡	our gadget shows it
Directory not randomized; no M/E→S delay	Two mechanisms the paper lists are absent from the build	suspected	code read (grep)
SpecLFB `amulet-SpecLFB`			
First speculative load unprotected (UV6)	The first speculative load is not held in the LFB and fills L1D	confirmed	11/12 tests in the strict window; 68/300 in a sweep
Second speculative load also unprotected	The queue head is never flagged and `PrevNoUnsafe` never clears, so the next load passes	strong candidate	code read; seen in one trace (`uv6x`)
Squashed load's LFB entry kept	A parked line is not dropped at the squash and can fill L1D later	suspected	code read; path never exercised
LFB refill reads a freed packet	The refill reuses a packet deleted when the line was parked	suspected	code read (`base.cc:838`)